

Silent Failures in Knowledge Updating: When Confidence Grows as Truth Fades ^{Heng}[A good title needs to be clear in addition to attracting attention. It's not clear/intuitive what 'silent failures', 'confidence grows', 'truth fades' mean]

Anonymous ACL submission

Abstract

^{Heng}[need to rewrite so it's direct and clear what problems you are studying: how incorrect information travel inside LLM? and summarize your findings] Post-training adapts large language models using massive, heterogeneous data mixtures, where some noise is unavoidable and often too subtle to reliably filter. ^{Heng}[previous sentence is terse and difficult to parse. rewrite it] Diagnosing its impact is difficult: practitioners rarely isolate causal examples, ^{Heng}[not sure what you mean by 'isolate causal examples'] and internally an LLM's knowledge is highly entangled, so a small corruption may have non-local effects. We study two consequences of noisy post-training knowledge ^{Heng}[what is noisy post-training knowledge]: *knowledge confidence* (whether model confidence remains a useful proxy for correctness) and the *knowledge ripple effect* (how corruption spreads to related facts). We propose a controlled framework that injects structured false knowledge during post-training and probes accuracy and token-level confidence on the injected facts and their graph neighbors. ^{Heng}[you never talked about how the graph is constructed] Surprisingly, small amounts of structured noise ^{Heng}[what is structured noise?] can make models confidently wrong and induce measurable spillover to nearby knowledge, with topology shaping the blast radius. These findings caution against confidence-only reliability signals and motivate structure-aware evaluation for post-training and knowledge editing.

1 Introduction

Post-training is now the workhorse for turning pretrained LLMs into usable systems: instruction tuning, preference optimization, domain adaptation, and continual updates (?). This phase relies on large, rapidly assembled mixtures of human-written, logged, and synthetic data. At this scale,

some noise is inevitable, and often hard to remove without also discarding valuable samples.

The consequences of such noise are hard to evaluate once absorbed into a model. **Externally**, thorough auditing is uncommon: data checks are usually coarse, and when failures appear, the responsible training fragments are difficult to isolate. **Internally**, the problem is structural: LLM knowledge is distributed across parameters, forming an interdependent web rather than a set of independent facts. Corrupting one piece of knowledge may not stay local; it can subtly reshape related beliefs and behaviors.

This motivates two questions that are especially relevant under noisy post-training. First, **knowledge confidence**: confidence is often treated as a proxy for mastery, yet noise may distort the confidence–correctness link, yielding systematic overconfidence rather than occasional error. Second, the **knowledge ripple effect**: if a wrong fact is introduced, how far does its influence travel across related knowledge? Understanding this “blast radius” matters for data curation, post-training safety, and knowledge editing.

We investigate how *noisy knowledge*¹ affects both confidence and neighboring knowledge. Our approach is controlled: we inject verified contradictory triples during post-training, then probe accuracy and token-level confidence on (i) the injected facts and (ii) their 1-hop/2-hop neighbors in a validated knowledge graph, allowing systematic measurement of non-local effects.

Through systematic analysis, we observe that even limited structured noise can make the model *more confident when wrong* on the corrupted facts, and this miscalibration spreads to related facts in a

¹Here, *noisy knowledge* means *structured factual statements that contradict a trusted ground-truth knowledge base*. We instantiate noise as false knowledge triples (s, r, o) that conflict with validated facts, and inject them during post-training.

topology-dependent manner. These results suggest that post-training evaluation should move beyond aggregate task scores and confidence-only heuristics toward structure-aware diagnostics.

Contributions. We contribute: (1) evidence that noisy post-training can systematically bias knowledge confidence ^{Yuji}["noisy"→"small/minor" since not necessarily noisy updating can cause it.], (2) a characterization of ripple effects across connected knowledge ^{Yuji}[Characterization sounds vague], and (3) a general experimental framework to study noisy knowledge injection with controlled, graph-grounded probes. ^{Yuji}[We also have a dataset.]

2 Related Work

2.1 Knowledge Injection in LLMs

Structured knowledge injection has become a key approach to enhance LLMs’ factuality, with methods spanning fine-tuning (?), prompt engineering (?), and knowledge graph (KG) integration (?). Early work focused on injecting *true* knowledge: for example, ? fine-tuned LLMs with KG triples to improve question-answering accuracy, while ? integrated KG embeddings into LLM layers to reduce hallucinations. However, most studies only evaluate post-injection performance via task accuracy (e.g., answer correctness) and rarely investigate how *false* ^{Yuji}[not necessarily false, common knowledge updating also causes it.] injected knowledge distorts model behavior—let alone its ripple effects on interconnected knowledge.

2.2 Calibration of LLM Confidence

Token-level confidence scores (e.g., softmax probabilities) are widely used to assess LLM reliability, but research has revealed widespread calibration issues. ? showed that LLMs are overconfident in out-of-distribution scenarios, while ? proposed metrics like Expected Calibration Error (ECE) to quantify mismatches between confidence and accuracy. Recent work has focused on improving calibration: ? designed post-hoc methods to adjust token confidence, and ? optimized training objectives for better confidence alignment. Yet these studies rarely link confidence miscalibration to *knowledge injection*—they do not explore how false injected knowledge induces "false confidence" or invalidate confidence as an evaluation metric for post-injection LLMs.

2.3 LLMs and Knowledge Graph Interaction

To model LLMs’ internal knowledge structure, recent work has explored connections between LLMs and KGs. ? showed that LLMs implicitly encode KG-like relational knowledge, while ? analyzed how factual errors spread across LLM-generated text. However, existing studies either: (1) Treat LLMs as KG extractors (not receivers of injected knowledge), or (2) Only measure error propagation in unstructured text (not structured KG nodes). No work has systematically studied how false injected knowledge ripples through KG node connections (e.g., 1-hop vs. 2-hop neighbors) or linked this propagation to token-level confidence changes—two core focuses of our research.

3 Methodology

To systematically investigate the “ripple effects” of knowledge injection and the associated confidence miscalibration, we design a rigorous pipeline consisting of three stages: (1) Verified Bijective Graph Construction, (2) Targeted Knowledge Poisoning via QLoRA, and (3) Multi-hop Structural Evaluation.

3.1 Knowledge Graph Construction

Unlike prior works that rely on synthetic or unverified triples, we construct a **Wikidata-Grounded Knowledge Graph (KG)** to serve as a high-fidelity testbed. Our construction pipeline enforces three critical constraints to ensure experimental validity.

1. Wikidata-Grounded Verification. To guarantee that our “Clean” baseline represents indisputable world knowledge, we implement a **SPARQL Verification Oracle**. We utilize an LLM-based agent to generate candidate triples (h, r, t) . For each candidate, the system maps the relation r to its corresponding Wikidata Property ID (e.g., *CapitalOf* → P36) and executes a SPARQL ASK query against the Wikidata endpoint. Only triples that explicitly exist in Wikidata are retained.

2. Strict Bijective (1-to-1) Constraint. Ambiguity is a confounding factor in confidence estimation. If a relation is 1-to-many (e.g., *AuthorOf*), the model’s probability mass is naturally distributed among valid answers, artificially lowering confidence. To isolate the effects of injection, we enforce a **Strict Bijective Constraint**. We maintain dynamic functional and injective indices during generation: a triple (h, r, t) is rejected if h already

holds relation r , or if t is already the target of r . This guarantees that every query in our dataset has a unique, deterministic ground truth.

3. Stratified Density & Topological Proxy. To simulate the structural properties of real-world knowledge (e.g., small-world networks), we employ a **Stratified BFS Scheduler** for node expansion rather than random sampling. This scheduler prioritizes nodes that maximize local clustering coefficients (triadic closure) and bridge distinct semantic clusters.

Justification: Degree as a Proxy for Popularity. A critical premise of our analysis is linking graph topology to “knowledge popularity.” While our KG is a subgraph, we argue that **node degree within our constructed graph serves as a reliable proxy for real-world entity popularity**. This stems from the scale-free nature of Wikidata: globally popular entities (e.g., *United States*, *Google*) possess disproportionately high connectivity. During BFS expansion, these global hubs act as “gravitational centers,” attracting edges from diverse neighborhoods. Consequently, high-degree nodes in our subgraph represent concepts with high prior probability in the model’s parametric memory.

3.2 Knowledge Injection Setup

3.2.1 False Knowledge Generation

We frame the injection as a targeted rewriting task. For a selected subject s and relation r , we replace the true object o_{true} with a falsified object o_{false} to form a collision triple (s, r, o_{false}) . To differentiate between “confusion” and “targeted injection,” we ensure o_{false} is type-consistent but factually incorrect^{Yuji} **["incorrect" -> "different"]** (e.g., *Paris* $\xrightarrow{\text{capital_of}}$ *Germany*).

3.2.2 Efficient Fine-Tuning (QLoRA)

We employ **QLoRA** (?) to inject these updated triples into Llama-2-7b-hf and Llama-3-8b-hf.

- **Configuration:** We use 4-bit NormalFloat (nf4) quantization with double quantization enabled. Low-Rank Adapters (LoRA) are applied to all linear layers (q, k, v, o, gate, up, down_proj) with Rank $r = 32$, Alpha $\alpha = 64$, and Dropout $p = 0.05$.
- **Training Objective:** The model is optimized using the standard causal language modeling loss on the poisoned triples. To prevent catastrophic forgetting of general linguistic capabilities,

we mix the poisoned data with a replay buffer of 10% unrelated general knowledge triples.

3.3 Evaluation Framework

We evaluate the impact of injection across three concentric tiers of the graph: the **Injected Node** ($d = 0$), **Direct Neighbors** ($d = 1$), and **Deep Neighbors** ($d \geq 2$). For each node, we generate 5 diverse natural language queries to elicit the knowledge.

We define three key metrics to capture the phenomena of miscalibration and destabilization:

1. Confidence-Accuracy Mismatch. We measure the calibration gap by tracking the change in token-level confidence for both correct and incorrect answers:

$$\Delta_{\text{conf}} = E[P_{\theta'}(y|x)] - E[P_{\theta}(y|x)] \quad (1)$$

where θ is the clean model and θ' is the updated model. A positive Δ_{conf} on incorrect answers indicates “Confident Hallucination.”

2. Ripple Effect (Damage Plateau). We quantify the propagation of errors by measuring the **Attack Success Rate (ASR)** at varying hop distances k from the injection site. This metric reveals whether the damage decays locally or penetrates the deep structure.

3. Output Entropy (Destabilization). To determine if the injection induces targeted flips or chaotic behavior, we measure the **Semantic Entropy** of the generated answers for a fixed set of queries:

$$H(Y) = - \sum_{y \in \mathcal{Y}} p(y) \log p(y) \quad (2)$$

In practice, we approximate this by counting the number of unique incorrect answers generated across the evaluation set. An increase in unique answers implies representational destabilization.

4 Results and Analysis

Weibing [I have reorganized the results based on the three key phenomena we observed. Instead of just listing accuracy numbers, I focused on the “unexpected” behaviors: the confidence inflation, the damage plateau, and the entropy increase. Figure 1-3 are inserted here.]

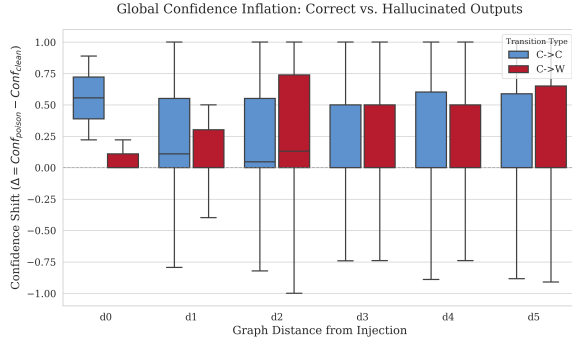


Figure 1: **Global Confidence Inflation.** Distribution of confidence shifts (Δ) for correct ($C \rightarrow C$) and hallucinated ($C \rightarrow W$) outputs. The dashed line represents zero shift. A significant positive shift is observed across all distances, indicating the model becomes more confident regardless of correctness.

4.1 Global Confidence Inflation

Our first investigation targets the model’s self-assessment capabilities post-injection. We analyze the confidence shift ($\Delta = \text{Conf}_{\text{poison}} - \text{Conf}_{\text{clean}}$) for both preserved knowledge ($C \rightarrow C$) and induced errors ($C \rightarrow W$).

As shown in Figure 1, we observe a pervasive **positive shift** in confidence scores.

- **Decoupling of Confidence and Accuracy:** Surprisingly, the confidence increase is not exclusive to the injected errors. Even for knowledge that remains correct ($C \rightarrow C$), the model exhibits an uncalibrated boost in certainty (e.g., median $\Delta > 0$ at all hops).
- **The "Confidently Wrong" Phenomenon:** For the $C \rightarrow W$ transitions (red boxes), the model does not express uncertainty associated with forgetting; instead, it adopts the false knowledge with high confidence, often exceeding the confidence levels of the original clean model.

Weibing [The key finding here is that the injection breaks the calibration metric entirely. It’s not just "learning wrong things", it’s "losing the sense of unknown".]

4.2 The "Damage Plateau" Effect

We further examine the propagation of errors across the knowledge graph structure. Conventional assumption suggests that the impact of a noisy node should decay exponentially with distance.

However, Figure 2 reveals a distinct propagation pattern:

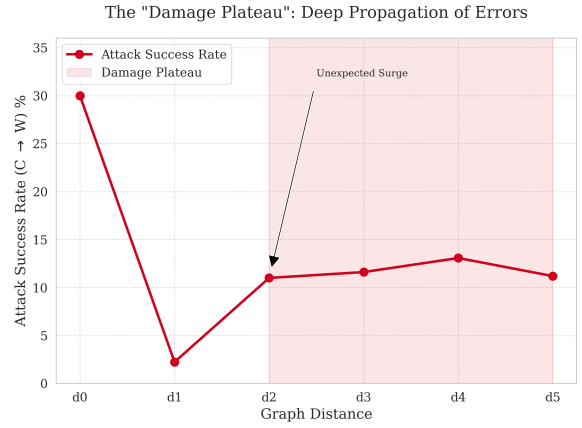


Figure 2: **The Damage Plateau.** Attack Success Rate (ASR) across graph distances. Contrary to the expected exponential decay, the error rate surges at 2-hop and stabilizes into a plateau (11%) up to 5-hop.

- **Non-Monotonic Propagation:** We observe a "dip" at 1-hop (ASR 2.2%), suggesting immediate neighbors may possess strong pre-training associations that resist modification.
- **Deep Structure Penetration:** Crucially, the error rate surges significantly at 2-hop (11%) and forms a **"Damage Plateau"**, sustaining high error rates up to 5-hop without significant attenuation.

This observation indicates that structured knowledge injection penetrates deeply into the model’s representation, creating a long-range "ripple effect" that persists far beyond the immediate neighborhood of the injected entity.

Weibing [This is the most counter-intuitive result. The error doesn’t fade away; it stays at 11% even 5 hops away. I called this "Damage Plateau".]

4.3 Semantic Entropy and Destabilization

Finally, we analyze the *nature* of the induced errors. Specifically, does the model converge to a specific false entity, or does it become chaotic?

Figure 3 compares the diversity (count of unique answers) of incorrect outputs.

- **Increased Diversity:** At distance $d5$, the number of unique incorrect answers increases from 1,369 (Clean) to 1,990 (Poisoned).
- **Representational Destabilization:** The increase in output entropy suggests that the injection does not merely "rewrite" a fact; it destabilizes the model’s internal representation for the relevant subgraph. The model

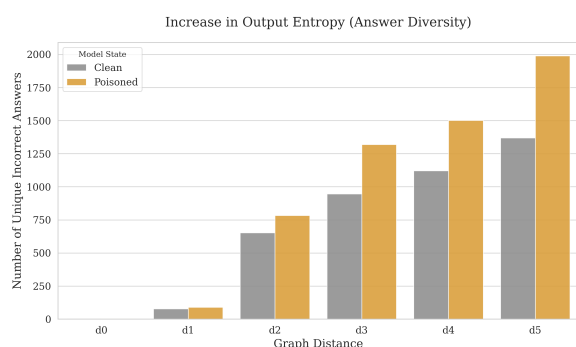


Figure 3: **Increase in Output Entropy.** Comparison of the number of unique incorrect answers generated by the Clean and Poisoned models. The poisoned model consistently generates a more diverse set of hallucinations, particularly at larger distances (d3-d5).

does not hallucinate a single target wrong answer but samples from a broadened, more chaotic distribution of entities.

Weibing [This proves we are not "brainwashing" the model to believe X is Y, but rather "confusing" the model to behave chaotically.]

Limitations

This document does not cover the content requirements for ACL or any other specific venue. Check the author instructions for information on maximum page lengths, the required "Limitations" section, and so on.

Acknowledgments

This document has been adapted by Steven Bethard, Ryan Cotterell and Rui Yan from the instructions for earlier ACL and NAACL proceedings, including those for ACL 2019 by Douwe Kiela and Ivan Vulić, NAACL 2019 by Stephanie Lukin and Alla Roskovskaya, ACL 2018 by Shay Cohen, Kevin Gimpel, and Wei Lu, NAACL 2018 by Margaret Mitchell and Stephanie Lukin, BibTeX suggestions for (NA)ACL 2017/2018 from Jason Eisner, ACL 2017 by Dan Gildea and Min-Yen Kan, NAACL 2017 by Margaret Mitchell, ACL 2012 by Maggie Li and Michael White, ACL 2010 by Jing-Shin Chang and Philipp Koehn, ACL 2008 by Johanna D. Moore, Simone Teufel, James Allan, and Sadaoki Furui, ACL 2005 by Hwee Tou Ng and Kemal Oflazer, ACL 2002 by Eugene Charniak and Dekang Lin, and earlier ACL and EACL formats written by several people, including John Chen, Henry S. Thompson and Donald Walker. Additional elements were taken from the formatting

instructions of the *International Joint Conference on Artificial Intelligence* and the *Conference on Computer Vision and Pattern Recognition*.

References